HANDS-ON TUTORIAL

From Selenium to Playwright

Understanding the `playwrightautomation` project & learning Playwright with JavaScript

Project: playwrightautomation — a 1:1 Playwright port of the actiTIME Selenium + TestNG framework

Application under test: actiTIME online demo (https://online.actitime.com)

Stack: Node.js · @playwright/test · Page Object Model · Excel-driven data

Audience: testers who know Selenium/TestNG and want to learn Playwright

Contents

1 · What this project is

This repository is a faithful re-implementation of an existing Selenium automation project, rebuilt with Playwright and JavaScript. The test *behaviour*, the locators, the test data and the flow are identical — only the automation framework changed.

1.1 · The original Selenium project

The source project (`Downloads/Project/Actitime`) is a classic Java test-automation framework built for practising UI automation against the actiTIME demo application. It uses:

- **Selenium WebDriver** — browser automation
- **TestNG** — test runner, annotations (`@BeforeClass` , `@BeforeMethod` …), and XML suite files
- **Apache POI** — reading test data from an Excel workbook
- **Page Object Model** — UI locators isolated in "page classes"

Its Java packages:

Class	Responsibility
`testbase.BaseClass`	Opens the browser (browser name read from Excel), navigates to the login page, logs in before every test, "logs out" after every test, closes the browser at the end.
`testbase.ExcelLibrary`	Helper methods `getExcelData(sheet, row, cell)` and `getLastRowNum(sheet)` .
`pageclasses.TasksPage`	Page object: `clickOnTasks()` , `clickOnAddNew()` .
`tasks.CreateCustomer`	Test: create a customer from Excel data.
`tasks.DeleteCustomer`	Test: search a customer and delete it permanently.
`users.CreateUser` / `users.DeleteUser`	Placeholder tests (they only print a line) — useful for demonstrating suite composition.

Four TestNG XML suites decide *which* tests run together: `smoke.xml` , `customer.xml` , `user.xml` , `functionlaity.xml` (spelling from the original).

1.2 · What changed in the port

Concept	Selenium + TestNG	Playwright
Language	Java	JavaScript (Node.js)
Browser driver	WebDriver + driver binaries	Bundled browsers, no drivers
Runner	TestNG	`@playwright/test`
Base class	`BaseClass` inheritance	A *fixture* file you import
Suite selection	XML files	Test **tags** + `--grep`
Excel	Apache POI	`exceljs` (+ a tiny sync reader for config)
Waiting	Implicit/explicit waits you write	Automatic — built into every action

> **KEY IDEA** Nothing about the *test intent* changed. If you understand the Selenium project, this document shows you the Playwright equivalent of each piece — that is the fastest way to learn Playwright.

2 · Playwright vs Selenium — the mental model shift

Four differences explain almost everything you will see in the code.

2.1 · Auto-waiting (the big one)

In Selenium you constantly fight timing: `WebDriverWait`, `ExpectedConditions`, `Thread.sleep`, `implicitlyWait`. In Playwright, **every action waits automatically** until the element is actionable — attached to the DOM, visible, stable (not animating), enabled, and not obscured by another element. Only then does it click/fill/etc.

```
// Selenium — you manage the wait
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(By.id("loginButton")))
    .click();

// Playwright — the wait is inside .click()
await page.locator('#loginButton').click();
```

That is why this project has almost no explicit waits, even though the Selenium original relied on a 30-second implicit wait.

2.2 · Locators are lazy and re-queried

A Selenium `WebElement` is a snapshot — grab it, and if the page re-renders you get a `StaleElementReferenceException`. A Playwright `Locator` is a *description* of how to find an element. It is resolved fresh every time you use it, so "stale element" simply does not exist.

```
const tasksLink = page.locator("a[href='/hni/tasks/tasklist.do']");
await tasksLink.click();   // re-finds the element right now
await tasksLink.click();   // re-finds it again — never stale
```

2.3 · Web-first assertions

Playwright's `expect()` for page/locator conditions **retries** until the condition is true or a timeout is hit. No manual polling.

```
// Retries for up to `expect.timeout` ms until the heading has this text
await expect(page.locator("h3")).toHaveText("Enter Time-Track for");
```

Compare with TestNG's `Assert.assertEquals(...)`, which checks exactly once.

2.4 · The test runner is included

Playwright ships its own runner: parallelism, retries, reporters, trace capture, fixtures, projects, and config — all in `@playwright/test`. There is no separate TestNG/JUnit layer.

TestNG	Playwright Test
`@Test`	`test('name', async () => { … })`
`@BeforeMethod`	`test.beforeEach(async () => { … })`
`@AfterMethod`	`test.afterEach(...)`
`@BeforeClass` / `@AfterClass`	`test.beforeAll` / `test.afterAll` (per file)
`@DataProvider`	a loop around `test(...)`, or a parametrised fixture
suite `.xml`	tags + `--grep`, or config `projects`
`Assert.*`	`expect(...)`

3 · Setup & installation

3.1 · Prerequisites

- **Node.js 18+** (this project was built on Node 25). Check: `node -v`
- **npm** (bundled with Node). Check: `npm -v`

3.2 · Install

```
# from the project root
npm install              # installs @playwright/test, exceljs, adm-zip
npx playwright install   # downloads the browser binaries (Chromium, Firefox, WebKit)
```

> **WHY TWO STEPS** `npm install` gets the npm packages. `npx playwright install` downloads the actual browsers Playwright drives. They are cached globally (`~/Library/Caches/ms-playwright`), not inside `node_modules` .

3.3 · First run

```
npm run test:user     # quickest: logs in, prints two lines, logs out
```

Expected output:

```
Running 2 tests using 1 worker

Create User
  ✓  1 [firefox] › tests/users/createUser.spec.js:4:1 › testCreateUser
Delete Uer
  ✓  2 [firefox] › tests/users/deleteUser.spec.js:4:1 › testDeleteUser

  2 passed
```

4 · Project structure

```
playwrightautomation/
├── package.json # scripts + dependencies
├── playwright.config.js # runner config; picks the browser from Excel
├── data/
│ └── data.xlsx # test data (verbatim copy of the Selenium project's)
├── src/
│ ├── testbase/
│ │ ├── excelLibrary.js # async Excel reader used by tests (ExcelLibrary.java)
│ │ └── excelSync.js # tiny sync Excel reader used only by the config
│ ├── pageclasses/
│ │ └── tasksPage.js # Page Object (TasksPage.java)
│ └── fixtures/
│ └── baseTest.js # custom `test` with login/logout (BaseClass.java)
├── tests/
│ ├── tasks/
│ │ ├── createCustomer.spec.js
│ │ └── deleteCustomer.spec.js
│ └── users/
│ ├── createUser.spec.js
│ └── deleteUser.spec.js
├── testng-suites-reference/ # the original .xml suites, kept for traceability
└── docs/
    └── tutorial.html / tutorial.pdf
```

Mapping to the Selenium project

Selenium (Java)	Playwright (JavaScript)
`com.krn.actitime.testbase.BaseClass`	`src/fixtures/baseTest.js`
`com.krn.actitime.testbase.ExcelLibrary`	`src/testbase/excelLibrary.js`
`com.krn.actitime.pageclasses.TasksPage`	`src/pageclasses/tasksPage.js`
`com.krn.actitime.tasks.CreateCustomer`	`tests/tasks/createCustomer.spec.js`
`com.krn.actitime.tasks.DeleteCustomer`	`tests/tasks/deleteCustomer.spec.js`
`com.krn.actitime.users.CreateUser` / `DeleteUser`	`tests/users/createUser.spec.js` / `deleteUser.spec.js`
`Data/data.xlsx`	`data/data.xlsx`
`smoke.xml` / `customer.xml` / `user.xml` / `functionlaity.xml`	tags `@smoke` `@customer` `@user` `@functionality` + npm scripts

5 · File-by-file walkthrough

5.1 · package.json

```
{
  "scripts": {
    "test":                "playwright test",
    "test:headed":         "playwright test --headed",
    "test:smoke":          "playwright test --grep @smoke",
    "test:customer":       "playwright test --grep @customer",
    "test:user":           "playwright test --grep @user",
    "test:functionality":  "playwright test --grep @functionality",
    "report":              "playwright show-report test-output"
  },
  "devDependencies": { "@playwright/test": "^1.55.0", "exceljs": "^4.4.0" },
  "dependencies":    { "adm-zip": "^0.5.18" }
}
```

Each `test:*` script is the equivalent of running one TestNG XML file. They all call the same `playwright test` binary with a different `--grep` tag filter.

5.2 · playwright.config.js

This is the runner's control panel. The interesting part is that it reproduces the Selenium `BaseClass` behaviour of choosing the browser from the `Browser` sheet of `data.xlsx`.

```
const { defineConfig } = require('@playwright/test');
const { getExcelDataSync } = require('./src/testbase/excelSync');

// driver.manage().window().maximize()  ->  a large desktop viewport
const MAXIMIZED = { width: 1920, height: 1080 };
const BROWSER_PROJECTS = {
  firefox:  { name: 'firefox',  use: { browserName: 'firefox',  viewport: MAXIMIZED } },
  chromium: { name: 'chromium', use: { browserName: 'chromium', viewport: MAXIMIZED } },
  chrome:   { name: 'chrome',   use: { browserName: 'chromium', channel: 'chrome', viewport:
MAXIMIZED } },
  // ie / edge  ->  msedge channel  (Playwright has no Internet Explorer)
  ie:       { name: 'msedge',   use: { browserName: 'chromium', channel: 'msedge', viewport:
MAXIMIZED } },
  webkit:   { name: 'webkit',   use: { browserName: 'webkit',   viewport: MAXIMIZED } },
};

let browserName = process.env.BROWSER || getExcelDataSync('Browser', 1, 0); // -> "firefox"
const project = BROWSER_PROJECTS[(browserName || 'chromium').toLowerCase()] ||
BROWSER_PROJECTS.chromium;

module.exports = defineConfig({
  testDir: './tests',
  fullyParallel: false,          // the Selenium project ran serially
  workers: 1,
  timeout: 60_000,               // per-test timeout
  expect: { timeout: 30_000 },     // mirrors the 30s implicit wait
  reporter: [['list'], ['html', { outputFolder: 'test-output', open: 'never' }]],
  use: {
    baseURL: 'https://online.actitime.com',
    actionTimeout: 30_000,
    navigationTimeout: 30_000,
    screenshot: 'only-on-failure',
    trace: 'on-first-retry',
  },
  projects: [project],
});
```

Option	What it does	Selenium equivalent
`testDir`	Where the runner looks for `*.spec.js`	—
`use.baseURL`	Prefix for `page.goto('/hni/login.do')`	`driver.get(fullUrl)`
`use.viewport`	Window size for the browser context	`window().maximize()`
`expect.timeout`	How long web-first assertions retry	`implicitlyWait(30s)`
`actionTimeout`	How long a click/fill waits for actionability	`implicitlyWait(30s)`
`trace`	Records a step-by-step debug trace	no equivalent — this is a Playwright superpower
`projects`	Named browser configurations to run	roughly: the browser `if/else` in `BaseClass`

> **WHY TWO EXCEL READERS?** `playwright.config.js` is loaded *synchronously* by the runner and must export a plain object. `exceljs` is async-only, so it cannot run here. `excelSync.js` is a ~50-line synchronous reader (an `.xlsx` is a zip; it unzips with `adm-zip` and parses the XML for one cell) used **only** to pick the browser at load time. Test code uses the richer async `excelLibrary.js`.

5.3 · src/testbase/excelLibrary.js

The port of `ExcelLibrary.java`. Apache POI uses 0-based row/cell indexes, so this keeps the same 0-based contract the tests were written against.

```
const ExcelJS = require('exceljs');
const DATA_FILE = path.join(__dirname, '..', '..', 'data', 'data.xlsx');

class ExcelLibrary {
  async getExcelData(sheetName, rowNum, cellNum) {
    const wb = new ExcelJS.Workbook();
    await wb.xlsx.readFile(DATA_FILE);
    const sheet = wb.getWorksheet(sheetName);
    if (!sheet) return null;
    const cell = sheet.getRow(rowNum + 1).getCell(cellNum + 1);  // POI 0-based -> exceljs
1-based
    return cell.value == null ? null : String(cell.value);
  }

  async getLastRowNum(sheetName) { /* returns sheet.actualRowCount - 1, like POI */ }
}
```

Usage inside a test: `const un = await xlib.getExcelData('Login', 1, 0);` — sheet `Login`, row index 1 (the second row), cell index 0 (first column).

5.4 · src/pageclasses/tasksPage.js — the Page Object Model

The Page Object Model (POM) keeps locators and page actions in one class so tests read as business steps, not CSS/XPath. This is the port of `TasksPage.java`. The only structural change: the Selenium version received `WebDriver` as a method argument; here the Playwright `page` is passed once to the constructor.

```
class TasksPage {
  constructor(page) {
    this.page = page;
  }

  async clickOnTasks() {
    await this.page.locator("xpath=//a[@href='/hni/tasks/tasklist.do']").click();
  }

  async clickOnAddNew() {
    await this.page.locator("xpath=//div[@class='addNewButton']").click();
  }
}
```

> **NOTE** `page.locator("xpath=…")` — the `xpath=` prefix tells Playwright the string is XPath. The original Selenium XPaths were kept verbatim so the port stays 1:1. In new Playwright code you would usually prefer `page.getByRole()`, `getByText()`, `getByLabel()`, or `getByTestId()` instead of XPath.

5.5 · src/fixtures/baseTest.js — fixtures & the test lifecycle

This file replaces inheritance from `BaseClass`. Instead of `extends BaseClass`, each spec does `const { test } = require('../../src/fixtures/baseTest')` and gets a `test` function that already knows how to log in and out.

```
const base = require('@playwright/test');
const { ExcelLibrary } = require('../testbase/excelLibrary');
const { TasksPage }    = require('../pageclasses/tasksPage');

// Add two fixtures: `xlib` and `tp`. Tests just list them as parameters.
const test = base.test.extend({
  xlib: async ({}, use) => { await use(new ExcelLibrary()); },
  tp:   async ({ page }, use) => { await use(new TasksPage(page)); },
});

test.beforeEach(async ({ page, xlib }) => {
  // --- BaseClass.openBrowser() ---
  await page.goto('/hni/login.do');
  await expect(page).toHaveTitle('actiTIME - Login');

  // --- BaseClass.login() ---
  const un = await xlib.getExcelData('Login', 1, 0);
  const pw = await xlib.getExcelData('Login', 1, 1);
  await page.locator('#username').fill(un);
  await page.locator('input[name="pwd"]').fill(pw);
  await page.locator('#loginButton').click();
  await expect(page.locator("xpath=//h3[text()='Enter Time-Track for']"))
        .toHaveText('Enter Time-Track for');
});

test.afterEach(async ({ page }) => {
  // --- BaseClass.logout() ---
  await page.locator("xpath=//button[@data-testid='popup_menu_button_profile']").click();
});

module.exports = { test, expect: base.expect };
```

What is a fixture?

A fixture is a named piece of setup that Playwright builds *on demand* and tears down automatically. If a test's callback lists `{ page, tp, xlib }`, Playwright:

1. creates a fresh browser `context` and `page` for that test,
2. constructs `tp` (a `TasksPage` bound to that page) and `xlib`,
3. runs `beforeEach` (login), then the test body, then `afterEach` (logout),

4. disposes the context — cookies, storage and the page are gone.

The built-in `page` fixture is what replaces `@AfterClass closeBrowser()`: you never close anything manually because each test gets an isolated context that Playwright cleans up.

LIFECYCLE MAPPING

TestNG	This project
`@BeforeClass openBrowser()`	config `projects` (browser choice) + `goto` / title check in `beforeEach`
`@BeforeMethod login()`	`test.beforeEach`
`@AfterMethod logout()`	`test.afterEach`
`@AfterClass closeBrowser()`	automatic context disposal

5.6 · The spec files

tests/tasks/createCustomer.spec.js

```
const { test } = require('../../src/fixtures/baseTest');

test('testCreateCustomer', { tag: ['@customer', '@functionality', '@smoke'] },
  async ({ page, tp, xlib }) => {
    await tp.clickOnTasks();
    await tp.clickOnAddNew();
    await page.locator("xpath=//div[text()='New Customer']").click();

    const custName = await xlib.getExcelData('CreateCustomer', 1, 0);  // "Prathap"
    const desc     = await xlib.getExcelData('CreateCustomer', 1, 1);  // "This is test"

    await page.locator("xpath=(//input[@placeholder='Enter Customer Name'])
[2]").fill(custName);
    await page.locator("xpath=//textarea[@placeholder='Enter Customer
Description']").fill(desc);
    await page.locator("xpath=//div[text()='Create Customer']").click();
  });
```

The second argument `{ tag: [...] }` is how a test joins one or more "suites". These three tags mean this test runs under `test:customer`, `test:functionality` and `test:smoke` — matching `customer.xml`, `functionlaity.xml` and `smoke.xml` in the original.

tests/tasks/deleteCustomer.spec.js

```
test('testDeleteCustomer', { tag: ['@customer', '@functionality'] },
  async ({ page, tp, xlib }) => {
    await tp.clickOnTasks();
    const custName = await xlib.getExcelData('DeleteCustomer', 1, 0);
    await page.locator("xpath=(//input[@placeholder='Quick search by name ...'])
[1]").fill(custName);
    await page.locator("xpath=//span[text()='" + custName +
"']/../../..//div[@class='editButton']").click();
    await page.locator("xpath=(//div[text()='ACTIONS'])[1]").click();
    await page.locator("xpath=(//div[text()='Delete'])[2]").click();
    await page.locator("xpath=//span[text()='Delete permanently']").click();
  });
```

tests/users/createUser.spec.js & deleteUser.spec.js

```
test('testCreateUser', { tag: ['@user', '@functionality', '@smoke'] }, async () => {
  console.log('Create User');
});
```

These do nothing but print — exactly like the Java originals. They still log in and out, because `beforeEach` / `afterEach` always run. Their value is to show how tags compose different suites.

6 · How the actiTIME flows work, step by step

6.1 · Every test (beforeEach → body → afterEach)

1. **Navigate** to `https://online.actitime.com/hni/login.do`.
2. **Assert** the page title is `actiTIME - Login`.
3. **Read** username + password from the `Login` sheet (`hemanthni101@gmail.com` / `Demo@123`).
4. **Fill** `#username` and `input[name="pwd"]`, **click** `#loginButton`.
5. **Assert** the dashboard heading reads `Enter Time-Track for`.
6. … the test body runs …
7. **Click** the profile menu button (the original's "logout" step).
8. Playwright disposes the context.

6.2 · Create Customer

1. `clickOnTasks()` → opens the Tasks list (`a[href='/hni/tasks/tasklist.do']`).
2. `clickOnAddNew()` → opens the "add new" dropdown (`div.addNewButton`).
3. Click `New Customer`.
4. Read `Prathap` / `This is test` from the `CreateCustomer` sheet.
5. Fill the customer name (2nd match of the placeholder) and description.
6. Click `Create Customer`.

6.3 · Delete Customer

1. `clickOnTasks()`.
2. Type the customer name into "Quick search by name …".
3. From the matching row, click its `editButton` (navigates from the name `span` up three ancestors, then down to the edit button).
4. Open `ACTIONS` → `Delete` → confirm `Delete permanently`.

> **REALITY CHECK** These locators are verbatim from the Selenium project. actiTIME's live UI evolves, so a step can break when the site's DOM no longer matches an old XPath (on the last run, `deleteCustomer` failed at the row-edit button for this reason). The *Playwright port is faithful*; fixing such a step means updating the locator, which is a great first exercise — see §9 (codegen) and §10.

7 · Data-driven testing with Excel

The workbook `data/data.xlsx` has four sheets. Row 0 is the header; row 1 holds the values the tests read.

Sheet	Col 0	Col 1	Row 1 value
`Login`	Username	Password	hemanthni101@gmail.com · Demo@123
`CreateCustomer`	Customer name	Description	Prathap · This is test
`DeleteCustomer`	Customer name	—	Prathap
`Browser`	Browser name	—	firefox

Reading a value:

```
const name = await xlib.getExcelData('CreateCustomer', 1, 0);
//                             sheet             row  col   (0-based, like Apache POI)
```

Turning it into a real data-driven test

`getLastRowNum()` exists so you can loop over every data row. Example pattern:

```
const xlib = new ExcelLibrary();
const last = await xlib.getLastRowNum('CreateCustomer');
for (let row = 1; row <= last; row++) {
  test(`create customer from row ${row}`, async ({ page, tp }) => {
    const name = await xlib.getExcelData('CreateCustomer', row, 0);
    // … same steps as createCustomer.spec.js …
  });
}
```

This is the Playwright equivalent of a TestNG `@DataProvider`.

8 · Running & debugging tests

8.1 · Everyday commands

Command	What it does
`npm test`	Run every test (headless)
`npm run test:smoke`	Only `@smoke` — CreateCustomer + CreateUser
`npm run test:customer`	Only `@customer` — CreateCustomer + DeleteCustomer
`npm run test:user`	Only `@user` — CreateUser + DeleteUser
`npm run test:functionality`	`@functionality` — all four
`npm run test:headed`	Same as `npm test` but shows the browser
`npx playwright test tests/tasks/createCustomer.spec.js`	One file
`npx playwright test -g "testCreateCustomer"`	Tests whose title matches
`BROWSER=chromium npm test`	Override the Excel browser choice
`npm run report`	Open the HTML report from the last run

8.2 · The three debugging superpowers

UI Mode — `npx playwright test --ui`

A visual runner: pick tests, watch them run, see each step, time-travel through DOM snapshots, inspect locators, and re-run on save. The best place to learn what the code is doing.

Debug Mode — `npx playwright test --debug`

Opens the Playwright Inspector and steps through the test line by line, with a "pick locator" tool.

Trace Viewer — `npx playwright show-trace`

This project sets `trace: 'on-first-retry'`. After a failing run:

```
npx playwright show-trace test-output/.../trace.zip
# or just open the HTML report — traces are linked from failures
```

A trace is a full recording: every action, before/after screenshots, network, console, and the DOM at each step. There is no Selenium equivalent.

Codegen — `npx playwright codegen https://online.actitime.com/hni/login.do`

Opens a browser, records your clicks and typing, and writes Playwright code with good locators. Use it to repair a broken step: perform the action by hand, copy the locator it suggests.

8.3 · Reports & artifacts

- **list reporter** — the live console output
- **html reporter** — `test-output/` ; open with `npm run report`
- **screenshots** — captured only on failure, attached to the report
- **traces** — captured on first retry

9 · Selenium → Playwright cheat sheet

Task	Selenium (Java)	Playwright (JavaScript)
Start browser	`new ChromeDriver()`	runner does it; you get `page`
Open URL	`driver.get(url)`	`await page.goto(url)`
Find element	`driver.findElement(By.id("x"))`	`page.locator('#x')`
By id	`By.id("username")`	`'#username'` or `page.getByLabel('Username')`
By name	`By.name("pwd")`	`'input[name="pwd"]'`
By XPath	`By.xpath("//h3[...]")`	`'xpath=//h3[...]'`
By text	`By.xpath("//div[text()='X']")`	`page.getByText('X', { exact: true })`
By role	—	`page.getByRole('button', { name: 'Login' })`
Nth match	`(//input[...])[2]`	`page.locator('input…').nth(1)`
Type text	`el.sendKeys("abc")`	`await loc.fill('abc')`
Click	`el.click()`	`await loc.click()`
Read text	`el.getText()`	`await loc.textContent()` / `innerText()`
Title	`driver.getTitle()`	`await page.title()`
Wait for element	`WebDriverWait + ExpectedConditions`	built in — actions auto-wait
Assert equals	`Assert.assertEquals(a, b)`	`expect(a).toBe(b)`
Assert visible text	get text, then assert	`await expect(loc).toHaveText('…')`
Assert title	get title, then assert	`await expect(page).toHaveTitle('…')`
Maximize	`driver.manage().window().maximize()`	`use: { viewport: { width, height } }`
Implicit wait	`manage().timeouts().implicitlyWait(...)`	`actionTimeout` / `expect.timeout` in config
Close	`driver.close()` / `quit()`	automatic per test
Screenshot	`((TakesScreenshot)driver)…`	`await page.screenshot({ path })` (auto on failure)

> **GOLDEN RULE** Prefer, in order: `getByRole` → `getByLabel` / `getByPlaceholder` / `getByText` → `getByTestId` → CSS → XPath. Role/label locators match what a user sees and survive redesigns far better than XPath.

10 · Exercises

Do these in order. Each teaches one Playwright idea.

1. **Watch it run.** `npx playwright test --ui`, run `@smoke`, and step through `testCreateCustomer`. Find where each Excel value is read.
2. **Read a trace.** Force a failure (change a locator to a wrong id), run the test, open the HTML report, and open the trace. Identify the exact failing step and the DOM at that moment.
3. **Modernise a locator.** In `tasksPage.js`, replace `xpath=//div[@class='addNewButton']` with a `getByRole` or `getByText` locator. Use `npx playwright codegen` to discover a good one.
4. **Add an assertion.** After creating a customer, assert a success toast or that the new customer name appears in the list — `await expect(page.getByText('Prathap')).toBeVisible()`.
5. **Repair `deleteCustomer`.** Open the Tasks page manually with codegen, search a customer, and record the real "edit / delete" locators. Update the spec so the whole `@customer` suite passes.
6. **Make it data-driven.** Add a second row to the `CreateCustomer` sheet and use the `getLastRowNum` loop from §7 so both customers are created.
7. **Add a cross-browser run.** Add a `projects` array in the config with `chromium`, `firefox` and `webkit`, and run `npx playwright test --project=webkit`.
8. **Speed it up.** Set `fullyParallel: true` and `workers: 4`. Observe that each test still logs in independently because contexts are isolated. (Bonus: cache login with `storageState` so `beforeEach` skips the UI login.)
9. **Replace the "logout" step.** The ported `afterEach` only opens the profile menu (faithful to the original bug). Make it actually click "Logout" and assert the login page is back.
10. **Convert to TypeScript.** Rename specs to `.spec.ts`, add `tsconfig.json`, and get autocomplete for `page`, `expect` and your page objects.

11 · Troubleshooting & FAQ

"Executable doesn't exist … run npx playwright install"

The browser binaries are not downloaded. Run `npx playwright install` (optionally `npx playwright install firefox chromium webkit`).

A step times out on a locator

The element isn't there, isn't visible, or the selector is wrong for the current site. Run with `--ui` or `--debug`, use the locator picker, or `codegen` to find the right selector. Timeouts come from `actionTimeout` / `expect.timeout` in the config.

Login fails

The demo credentials in `data/data.xlsx` (`Login` sheet) may have expired or the account state changed. Update the sheet with working actiTIME demo credentials.

"deviceScaleFactor is not supported with null viewport"

Don't combine a spread `devices[...]` preset with `viewport: null`. This project sets an explicit numeric `viewport` instead.

Config error: "must export a single object"

`playwright.config.js` must `module.exports` a plain object (or a `defineConfig(...)` result) — not a function/promise in this Playwright version. That is why the browser is read with the synchronous `excelSync.js`.

Why is `adm-zip` a normal dependency, not dev?

`playwright.config.js` uses it at load time via `excelSync.js`. Everything else (the runner, `exceljs`) is dev-only.

Where are screenshots / videos / traces?

Under `test-results/` (per-test artifacts) and linked from the HTML report in `test-output/`. Both are git-ignored.

How do I run just one test while developing?

Add `.only`: `test.only('testCreateCustomer', …)`, or use `npx playwright test -g "testCreateCustomer"`.

12 · Where to go next

- **Official docs** — playwright.dev/docs (start with "Writing tests", "Locators", "Assertions", "Trace viewer", "Test fixtures").
- **Best practices** — playwright.dev/docs/best-practices.
- **Locators guide** — playwright.dev/docs/locators.
- **API reference** — Page, Locator, expect.
- **VS Code extension** — "Playwright Test for VSCode": run/debug from the gutter, pick locators, record new tests.
- **Auth caching** — playwright.dev/docs/auth to skip UI login with `storageState`.

You now have the full map: this project is the Selenium actiTIME framework with the timing pain removed, inheritance replaced by fixtures, and XML suites replaced by tags. Work through the exercises in §10 and you will know Playwright.